CS3500 LAB3 REPORT

Name: Ashish Cherian Thannickal

Roll No: CS23B099

Problem 1 - Page Table Tree Printing in xv6

Changes made/Code added –

1. kernel/vm.c – completed the function void vmprint(pagetable_t pagetable) along with helper functions vmprintwalk(pagetable_t pagetable, int level) and printindent(int level).

```
//This code was added by Ashish CS23B099
void printindent(int level){
  for(int i = 0; i < level; i++)
    printf(" ..");
}

void vmprintwalk(pagetable_t pagetable, int level){
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if(pte & PTE_V){
      printindent(level);
      printf("%d: pte %p pa %p\n", i, (void*)pte, (void*)PTE2PA(pte));
      if((pte & (PTE_R|PTE_W|PTE_X)) == 0){
        // this PTE points to a lower-level page table.
        uint64 child = PTE2PA(pte);
        vmprintwalk((pagetable_t)child, level+1);
      }
    }
  }
}

void
vmprint(pagetable_t pagetable) {
  // your code here
  printf("page table %p\n", (void*)pagetable);
  vmprintwalk(pagetable, 1);
}
//Added code ends here
```

2. kernel/exec.c – inserted a call to vmprint in the function int exec(char* path, char** argv) to print the page table for the first user process(pid = 1).

```
//This code was added by Ashish CS23B099
if(p->pid == 1){
  vmprint(p->pagetable);
}
//Added code ends here
```

3. kernel/defs.c – function prototype for vmprint was already present.

Logic/ Working –

- The vmprint function follows almost exactly the same logic as the freewalk function present in the vm.c file. It goes through each pagetable entry in a pagetable(starting from the root pagetable) and if the entry is valid it prints the entry in the required format.
- For every entry, if the RWX bits are all 0 then the entry is a pointer to a lower level page table and we recursively use that pagetable as root for vmprintwalk.
- This continues until we reach a leaf node where the entry is printed in the required format and the function returns to the caller.

Problem 2 - Optimizing the getpid() System Call in xv6 via Page Table Sharing

Changes made/Code added –

1. kernel/proc.h – Added the field struct usyscall* usyscall to the proc struct in proc.h

```
struct usyscall *usyscall;   // This line was added by Ashish CS23B099
```

2. kernel/proc.c – Made the following changes in the following functions :

- static struct proc*allocproc(void) : allocated space for the read only page that can be accessed by both kernel and userspace. Also initialized the pid field to

the process pid.

```
//This code was added by Ashish CS23B099
if((p->usyscall = (struct usyscall *)kalloc()) == 0){
  freeproc(p);
  release(&p->lock);
  return 0;
}
p->usyscall->pid = p->pid;
//Added code ends here
```

- static void freeproc(struct proc *p): used kfree in order to free up the space used by the common page at USYSCALL. Followed the same pattern used by the trampoline page

```
//This code was added by Ashish CS23B099
if(p->usyscall)
  kfree((void*)p->usyscall);
p->usyscall = 0;
//Added code ends here
```

- pagetable_t proc_pagetable(struct proc *p): map the shared read only page to the USYSCALL location using mappages. Followed the same pattern as used for the trampoline page.

```
//This code was added by Ashish CS23B099
  if(mappages(pagetable, USYSCALL, PGSIZE,
              (uint64)(p->usyscall), PTE_R | PTE_U) < 0){
  uvmunmap(pagetable, TRAMPOLINE, 1, 0);
  uvmunmap(pagetable, TRAPFRAME, 1, 0);
  uvmfree(pagetable, 0);
  return 0;
}
//Added code ends here
```

- void proc_freepagetable(pagetable_t pagetable, uint64 sz): Added a line to unmap the part of the pagetable pointed to be USYSCALL.

```
uvmunmap(pagetable, USYSCALL, 1, 0); // This line was added by Ashish CS23B099
```

Logic/ Working –

- The code was implemented very similarly to the code for the trapframe page.

- Whenever a new process is created, allocproc is called and it finds an unused proc and allocates trapframe, and a process pagetable. The modified code also allocates space for the usyscall and initializes it with the pid for the current process.
- Then proc_pagetable maps the virtual addresses for the pages in allocproc. Here we are mapping USYSCALL to a read-only userspace page using mappages .
- This page is now accessible from user mode and thus we can find pid more efficiently without calling an interrupt/ going to kernel to get the pid.
- The USYSCALL page is also freed similar to the trapframe page when the program terminates.

Problem 3 - Detecting Accessed Pages: Implementing pgaccess() in xv6

Changes made/Code added –

1. kernel/syscall.h – Added the new syscall number for SYS_pgaccess and set it as 35.

```
#define SYS_pgaccess  35 // This line was added by Ashish CS23B099
```

2. kernel/syscall.c – added the entry for the syscall as well as added it to the array of function pointers.

```
extern uint64 sys_pgaccess(void); // This line was added by Ashish CS23B099
```

```
[SYS_pgaccess] sys_pgaccess, // This line was added by Ashish CS23B099
```

3. user/user.h – added the function prototype for pgaccess.

```
int pgaccess(void* base, int len, void* mask); // This line was added by Ashish CS23B099
```

4. user/usys.pl – added an entry for pgaccess so usys.S creates the relevant stub.

```
entry("pgaccess"); #This line was added by Ashish CS23B099
```

5. kernel/riscv.h – added a macro for PTE_A.

```
#define PTE_A (1L << 6) // This line was added by Ashish CS23B099
```

6. kernel/sysproc.c – added the function definition for uint64 sys_pgaccess(void).

```c
uint64
sys_pgaccess(void){
  uint64 base;
  int len;
  uint64 mask;
  argaddr(0,&base);
  argint(1,&len);
  argaddr(2,&mask);

  if(len > 32) // since pgaccess return an int, keep len atmost 32
    len = 32;

  struct proc *p = myproc();
  unsigned int abits = 0; //no bits are assigned initially
  for(int i = 0; i < len; i++){
    pagetable_t pagetable = p->pagetable; //getting the process pagetable
    uint64 va = base + i * PGSIZE; //finding the corresponding virtual address
    pte_t *pte = walk(pagetable, va, 0); //getting the pte for the virtual address
    if(pte && (*pte & PTE_V) && (*pte & PTE_A)){ //ensure pte is not null and then check for valid and accessed bit
      abits |= (1 << i);
      *pte &= ~PTE_A; // set accessed bit to 0;
    }
  }
  sfence_vma(); //flush the TLB
  if(copyout(p->pagetable,mask,(char*)&abits,sizeof(abits)) < 0)
    return -1;
  return 0;
}
```

Logic/ Working –

- set up a syscall pgaccess similar to trace for lab2.
- Use the argint and argaddr to get the three arguments for the syscall. Keep a bound on len (32 in this case since the).
- Now go through the virtual addresses in the range and find the entry corresponding to that address(using the walk function) and if it was accessed, add it to the bitmask and reset the accessed bit to 0, so that future calls to pgaccess do not include the same page.
- Flush the TLB using sfence_vma() as a precaution.
- Now after the bitmask is constructed, using the copyout function copy the bitmask to specified location in the user space (mask).